Ders 12 Genişletilmiş Notu: Kod Kusuru Analizi Sorusunun Çözüm Yöntemi ve Video Kiralama Örneğinin Son Tasarımı

Ders 12 Genişletilmiş Notu: Kod Kusuru Analizi Sorusunun Çözüm Yöntemi ve Video Kiralama Örneğinin Son Tasarımı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Oturumun Amacı
- 2. Örnek Kod: fowler_00 Customer Sınıfı
- 3. Kod Kusurlarını İşaretleme ve Eylemleri Gerekçelendirme
- 4. Son Tasarımın Sınıf Şemasıyla Gösterilmesi
- Örnek Sorular

1. Oturumun Amacı

Bu kısa oturum dönemin son buluşmalarından biridir. Hoca girişte dersin konularının bittiğini söyledi: kod kusurları (code smell) ve refactoring eylemlerinin hepsi anlatılmış, Fowler'ın video kiralama örneği de baştan sona müdahale müdahale yürütülmüştü. Bu yüzden yeni bir konu işlenmedi. Oturumun akademik kısmı, kod kusuru analizi türündeki bir sorunun nasıl çözüleceğinin Fowler örneğinin ilk sürümü (`fowler_00`) üzerinde gösterilmesidir.

Soru türü şöyledir: öğrenciye bir iki sayfalık bir Java kodu verilir. Öğrenciden üç şey beklenir:

- Kodda hangi kod kusurlarının nerede görüldüğünü işaretlemek.
- Her kusuru gidermek için hangi refactoring eyleminin yapılması gerektiğini yazmak.
- Bütün eylemler uygulandıktan sonra varılacak en doğru tasarımı bir UML sınıf şemasıyla göstermek.

Hoca, bir sınavda bu eylemlerin adım adım yürütülmesinin beklenmediğini, ama eylemlerin hepsi uygulandığında tasarımın nereye varacağının bilinmesi gerektiğini belirtti.

2. Örnek Kod: fowler_00 Customer Sınıfı

Hoca Eclipse'teki `fowler_00` paketinin kodunu bir kelime işlemciye kopyalayıp tek sayfaya sığacak biçimde küçülttü. Kodun büyük kısmı `Customer` sınıfındadır; `Movie` ve `Rental` sınıfları bu sürümde görece küçüktür. Ekrandaki `Customer` sınıfı şudur (ekrandan, 00:10:55 ve 00:14:40):

```
package fowler_00;

import java.util.*;

public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){ _name = name; }
```

```
    public void addRental(Rental arg) { _rentals.addElement(arg); }
    public String getName (){ return _name; }
    public String statement( ) {
        double totalAmount = 0;
        int frequentRenterPoints = 0;
        Enumeration<Rental> rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";
        while (rentals.hasMoreElements()) {
            double thisAmount = 0;
            Rental each = rentals.nextElement();
            //determine amounts for each line
            switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2.0;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3.0;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
            }
            // add frequent renter points
            frequentRenterPoints ++;
            // add bonus for a two day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                    && each.getDaysRented() > 1) frequentRenterPoints ++;
            //show figures for this rental
            result += "\t" + each.getMovie().getTitle()+ "\t" +
                    String.valueOf(thisAmount) + "\n";
            totalAmount += thisAmount;
        }
        //add footer lines
        result += "Amount owed is " + totalAmount+"\n";
        result += "You earned " + frequentRenterPoints
                + " frequent renter points";
        return result;
    }
}
```

Hoca kodu sayfaya sığdırmak için tek satırlık gövdeleri (yapıcı, `addRental`, `getName`) tek satıra taşıdığını, böyle küçük biçim değişiklikleri yapılabileceğini belirtti. Kod derste yeniden yazılmadı, yalnız gösterildi; bu yüzden burada derlenmedi.

3. Kod Kusurlarını İşaretleme ve Eylemleri Gerekçelendirme

3.1 Her kusura karşılık bir eylem

Hoca cevabın "şuradaki şu kod kusuruna karşılık şu refactoring eylemi" biçiminde kurulmasını istedi. Göstermek için Zoom'un çizim aracını kalem gibi kullanarak kodun üstünde iki işaretleme yaptı ve sayfanın altına şu iki satırı yazdı (ekrandan, 00:14:40):

```
#1: long method. İşlem: Extract method.
#2: long method. İşlem: Replace temp w. query.
```

- `#1`, `statement()` metodunun imzası çevrelenerek verildi. Teşhis Long Method, çözüm Extract Method'dur.

- #2, döngü içindeki geçici değişken için verildi. Hoca bunun da bir Long Method olduğunu, ama geçici değişken kullanımının teşvik ettiği bir uzun metot olduğunu söyledi. Çözüm Replace Temp with Query'dir.

3.2 Gerekçe ve niyet yazmak

Kusurun ve eylemin adı tek başına yeterli değildir. Hoca her işaretleme için bir iki cümlelik açıklama beklediğini söyledi ve iki örnek verdi:

- #1 için: "Bu metot diğer bütün metotlardan çok uzundur ve içinde birçok alt iş barındırmaktadır. Bu metodun parçalanması gerekmektedir." Buna ek olarak niyet de yazılmalıdır: çıkarılan şu metodun şu sınıfa taşınması daha yararlı olacaktır, ya da bu metodun aynı sınıfta kalması daha yararlıdır.
- #2 için: "Buradaki geçici değişken kodun uzamasını teşvik etmektedir. Bu metodun zaten kısaltılmaya ihtiyacı vardı. Bu değişkeni Replace Temp with Query eylemiyle ortadan kaldırmak, metodun kısaltılmasını kolaylaştırır."

  Ek bilgi (derste anlatılmadı): Replace Temp with Query, değeri bir ifadeyle hesaplanan yerel değişkenin yerine o ifadeyi döndüren bir sorgu metodu koyar. Yerel değişken kalmayınca kodun parçaları Extract Method ile daha kolay ayrılır, çünkü çıkarılacak parçanın taşıması gereken yerel değişken sayısı azalır. Ders 10'da `thisAmount` yerine `aRental.getCharge()` çağrısının yazılması bu eylemdi.

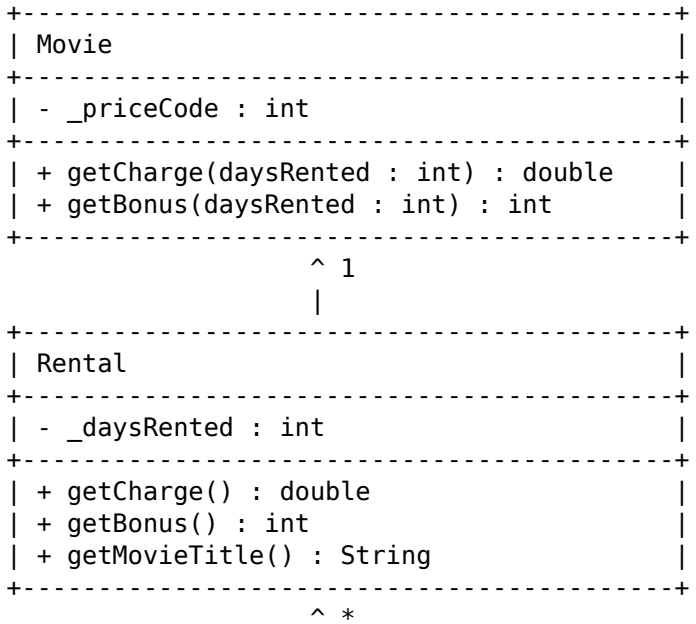
3.3 İşaretleme tekniği

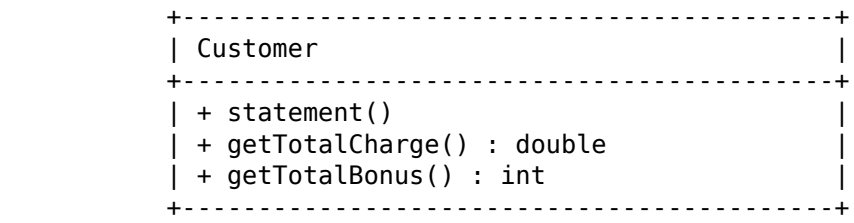
Hoca iki yol gösterdi: numarayı, kusur adını ve eylem adını kodun üstüne yazıp açıklamayı ayrı bir yere yazmak; ya da kodun üstünde yalnız numara vermek ve "numara, kusur, eylem, gerekçe" bilgisinin hepsini ayrı yere yazmak. Her şeyi kodun üstüne yazmak sayfayı sıkıştırır, hem okumayı hem yazmayı zorlaştırır. Hocanın önerdiği yol ikincisidir: kodda numaralı işaretler, ayrı yerde "şurada şu var, çözümü şu" biçiminde bir iki cümlelik açıklamalar.

> **Hocanın vurgusu:** Her kod kusuruna karşılık hangi refactoring eyleminin yapılacağı, kısa bir gerekçeyle birlikte yazılmalıdır; yalnız ad vermek yetmez.

4. Son Tasarımın Sınıf Şemasıyla Gösterilmesi

Bütün eylemler uygulandıktan sonra tasarımın son hali kodla değil, UML sınıf şemasıyla gösterilir. Hoca bunun için ders notlarındaki video kiralama örneğinin 9. müdahale sonucunu açtı (ekrandan, 00:17:10, slayt 323). Şemada metotların hangi sınıfa gittiği imzalarıyla görünür:





Bu tasarımda sorumluluklar veriye göre dağıtılmıştır. Fiyat kodu `Movie`'de olduğundan ücret ve bonus hesabı `Movie`'dedir ve kiralama gün sayısını parametre olarak alır. `Rental` gün sayısını bilen sınıf olarak `getCharge()` ve `getBonus()` çağrılarını kendi `_daysRented` değeriyle `Movie`'ye iletir ve film adını `getMovieTitle()` ile sunar; böylece `Customer`, `Movie`'ye doğrudan erişmez. `Customer` makbuzu `statement()` ile, toplamları `getTotalCharge()` ve `getTotalBonus()` ile üretir.

Hoca, sınıflar yazılırken ayrıntının önemli olduğunu belirtti: çıkarılıp yerleştirilen metotlar hangi sınıfa gittiyse o sınıfta imzasıyla görünmelidir. Örneğin `Rental` `getCharge`, `getBonus` ve `getMovieTitle` metotlarını, `Movie` de `getCharge` ve `getBonus` metotlarını şemadaki imzalarla kazanmıştır. Şema UML gösterim kurallarına uygun çizilmelidir. Aynı slaytta bir sıralama şeması da vardır, ama hoca son hali göstermek için sıralama şemasının gerekmediğini söyledi. Slayttaki sıralama şemasının akışı şöyledir (ekrandan, 00:17:10):

```
Kullanıcı -> aCustomer : statement()
  [her kiralama için]
    aCustomer -> aRental : getMovieTitle()   ; aRental -> aMovie : getTitle()
    aCustomer -> aRental : getCharge()       ; aRental -> aMovie : getCharge(_daysRented)
    (not: tek kira için ücret hesabı ve makbuz satırı basımı tamamlanır)
  aCustomer -> aCustomer : getTotalCharge()
    [her kiralama için] aCustomer -> aRental : getCharge() ; aRental -> aMovie : getCharge(_da
  aCustomer -> aCustomer : getTotalBonus()
    [her kiralama için] aCustomer -> aRental : getBonus()  ; aRental -> aMovie : getBonus(_day
  (not: makbuz basımı tamamlanır)
```

> **Hocanın vurgusu:** Son hali göstermek için kod yazılmaz, sınıf şeması kullanılır.

Örnek Sorular

Soru 1 (ekrandan, 00:10:55 ve 00:14:40): Yukarıdaki `fowler_00.Customer` kodunda hangi kod kusurlarını nerede görüyorsunuz? Bunları gidermek için hangi refactoring eylemleri yapılmalıdır? Bütün eylemlerden sonra varılacak en doğru tasarımı UML sınıf şemasıyla gösterin.

> **Derste verilen kısmi çözüm:** #1 `statement()` → Long Method → Extract Method (metot çok uzun ve birçok alt iş barındırıyor, parçalanmalı; çıkarılan metotların nereye taşınacağı belirtilmeli). #2 döngüdeki geçici değişken → Long Method → Replace Temp with Query (değişken metodun uzamasını teşvik ediyor; kaldırılması metodun kısaltılmasını kolaylaştırır). Son tasarım: bölüm 4'teki `Movie`–`Rental`–`Customer` sınıf şeması. Hoca diğer kusurları bu oturumda saymadı; geri kalan eylemler ders 10'da yürütülen müdahalelerdir.

Dersin Toparlaması

Bu oturumda yeni bir konu işlenmedi; dönem boyunca öğrenilen kod kusuru ve refactoring bilgisinin bir soruda nasıl kullanılacağı gösterildi. Yöntem üç adımdır: kodda kusuru numarayla işaretlemek, her kusur için uygulanacak refactoring eylemini bir iki cümlelik gerekçe ve niyetle yazmak, son tasarımı imzaları içeren bir UML sınıf şemasıyla çizmek. Örnek olarak Fowler'ın video kiralama kodunun ilk sürümünde `statement()` için Extract Method ve döngüdeki geçici değişken için Replace Temp with Query gösterildi; varılan son tasarım ise 9. müdahale sonundaki, hesabın `Movie`'de, iletimin `Rental`'da, toplamların ve makbuzun `Customer`'da olduğu yapıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.